



ENHANCING COMPUTER SYSTEM PERFORMANCE THROUGH EFFICIENT PROCESSOR AND MEMORY DESIGN

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1220 – Computer Architecture

to the award of the degree of

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

Vishwa.U (Reg. No: 192511192)

Under the Supervision of

Radhika.S

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Enhancing Computer System Performance through Efficient Processor and Memory Design" has been carried out by Vishwa U (Reg. No: 192511192) under the supervision of Radhika.S and is submitted in partial fulfilment of the requirements for the current semester of the B.E Computer Science and Engineering programme, under the course CSA1220 – Computer Architecture, at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. V. Sarala
Professor, Department of CSE,
SIMATS Engineering, Chennai – 602105.

COURSE FACULTY

Radhika.S
Department of CSE,
SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, Vishwa U, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Enhancing Computer System Performance through Efficient Processor and Memory Design" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: September 2026

Signature of the Student with Name: Vishwa.U

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Sections	Approx. Contribution (%)
Vishwa.U – 192511192	Responsible for the complete architectural design, simulation implementation, and overall development of the project	Complete pipeline datapath design, cache-hierarchy modelling, branch-predictor and prefetcher implementation, simulator integration	Complete benchmark execution, IPC/CPI/AMAT analysis, verification against reference model, debugging, result validation	All sections	100%

ABSTRACT

Modern computing performance is increasingly limited not by processor clock speed alone but by the widening gap between processor and main-memory speeds, commonly termed the "memory wall", together with pipeline stalls caused by data, control, and structural hazards. This capstone project presents PMOpt, a configurable processor-memory co-design and performance-analysis framework that models a 5-stage pipelined RISC-style processor with hazard detection and operand forwarding, a two-level cache hierarchy (split L1 instruction/data caches and a unified L2 cache), a dynamic tournament branch predictor, and a stride-based hardware prefetcher, and evaluates their combined effect on system performance under representative benchmark workloads.

The need for the project arises from the well-documented observation that DRAM access latency has improved far more slowly than processor clock frequency over recent decades, so that an unoptimised processor-memory design leaves substantial performance on the table regardless of core micro-architecture quality. PMOpt addresses this using a three-part architecture: a pipelined execution core with a hazard-detection and forwarding unit to minimise stall cycles; a two-level, configurable cache hierarchy sized and tuned to reduce the Average Memory Access Time (AMAT); and a prediction/prefetch layer combining a tournament branch predictor with a stride prefetcher to reduce control-hazard penalties and hide memory latency ahead of demand.

The methodology followed a structured engineering design process: requirements elicitation from computer-architecture literature and course objectives, a weighted trade-off study across three architectural alternatives, detailed datapath and cache-hierarchy design with supporting engineering calculations, and a verification test plan executed on a cycle-accurate simulator built and cross-checked against an instruction-accurate reference model, using a five-kernel benchmark suite (matrix multiplication, quicksort, FFT, string search, and 2-D convolution) totalling approximately 50 million dynamic instructions. Results demonstrate an average Cycles-Per-Instruction (CPI) of 1.32 ($\text{IPC} \approx 0.76$) against a non-pipelined, cache-less baseline CPI of 4.80, a $3.64\times$ overall speedup, an L1 cache hit rate of 96.4% and L2 hit rate of 91.2%, an AMAT reduced to 2.3 cycles, and a branch-prediction accuracy of 93.6%, at an estimated dynamic-power overhead of only 14% relative to a pipeline-only design.

The design was additionally assessed for sustainability (energy-delay product improvement), ethics (disclosed and appropriately licensed benchmark and tooling use), and safety/reliability (cross-validation of the cycle-accurate model against a golden reference simulator to prevent propagation of erroneous performance conclusions). The project concludes that a co-designed pipeline and memory hierarchy, evaluated through a systematic, weighted engineering trade-off process, can deliver substantial, measurable system-performance gains over a naïve baseline, and identifies extension to multi-core cache coherence and out-of-order execution as the principal directions for future work.

Keywords: Computer architecture, processor pipelining, memory hierarchy, cache optimisation, branch prediction, performance evaluation, computer organization.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	5
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	9
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion, Future Work and Reflection	24
	References	27
	Appendices	29

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Conceptual view of the processor–memory performance gap ("memory wall")	3
3.1	System architecture of PMOpt (pipeline core, hazard/forwarding unit, two-level cache, predictor & prefetcher)	12
3.2	5-stage pipeline datapath with operand-forwarding paths	13
3.3	Cache-hierarchy and branch-prediction data flow (IF → ID → EX → MEM → WB with L1/L2 and predictor)	14
4.1	Simulation and benchmark-analysis toolchain	18
4.2	IPC and CPI comparison: baseline vs. alternative designs across benchmarks	21
4.3	Cache hit-rate and AMAT comparison across design alternatives	22
4.4	Project Gantt chart / milestone timeline	23

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing processor/memory design approaches	7
3.1	Stakeholder / user requirements	9
3.2	Functional and non-functional requirements	10
3.3	Applicable standards and codes	10
3.4	Design specifications	11
3.5	Alternative-solution evaluation matrix	12
3.6	Trade-off analysis	15
3.7	Sustainability, ethics, safety, risk & security evaluation	16
3.8	Risk register	16
4.1	Tools, software and equipment used	18
4.2	Test plan and acceptance criteria	19
4.3	Verification results summary	20
4.4	Objective achievement matrix	22
4.5	Roles, responsibilities and budget	23

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
IPC	Instructions Per Cycle	—
CPI	Cycles Per Instruction	—
AMAT	Average Memory Access Time	cycles
ISA	Instruction Set Architecture	—
L1 / L2	Level-1 / Level-2 Cache	—
BTB	Branch Target Buffer	entries
LRU	Least Recently Used (replacement policy)	—
RISC	Reduced Instruction Set Computer	—
EDP	Energy-Delay Product	—
DRAM	Dynamic Random-Access Memory	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Processor performance has historically been assumed to scale with clock frequency, yet since the early 2000s the dominant limiter of realised system performance has shifted to the "memory wall" — the growing gap between processor cycle time and DRAM access latency — compounded by pipeline stalls arising from data, control, and structural hazards inside the processor core itself. A poorly designed pipeline or an unoptimised memory hierarchy can leave a processor stalled for the majority of its cycles even when its peak clock speed is high, so that overall Cycles-Per-Instruction (CPI) is dominated by memory-access and hazard-resolution penalties rather than by the intrinsic speed of the arithmetic/logic datapath. This project, undertaken under the course CSA1220 – Computer Architecture, uses architectural (cycle-accurate) simulation rather than physical fabrication to design and quantitatively evaluate a processor–memory co-design that reduces this gap, reflecting a realistic computer-architecture design-and-evaluation workflow.

1.2 Need for the Project

- Why the problem needs to be addressed: A large fraction of the performance headroom of modern applications is lost to pipeline stalls and memory-access latency rather than to insufficient raw compute throughput.
- Who is affected: Systems architects and embedded/mobile SoC designers (design-space exploration cost), application developers (unpredictable performance on memory-bound workloads), and end users (slower, less energy-efficient devices).
- Existing limitations: Simple in-order, cache-less processor models used in introductory coursework do not reflect the performance behaviour of real systems; commercial superscalar/out-of-order processors are accurate but far too complex and power-hungry to explore at course scale.
- Engineering significance: The problem requires jointly reasoning about pipeline hazard-handling, multi-level cache sizing/associativity, branch prediction, and prefetching under explicit performance, power, and design-complexity constraints — a representative computer-architecture design-space-exploration problem.

1.3 Problem Statement

Design and evaluate an efficient processor pipeline and memory-hierarchy configuration — combining hazard-aware pipelining, a multi-level cache hierarchy, dynamic branch prediction, and hardware prefetching — that maximises system throughput (IPC) while minimising average memory-access latency (AMAT) and estimated dynamic power, under given design-complexity and area constraints, validated through cycle-accurate simulation across a

representative benchmark workload suite. The problem involves multiple interacting factors — pipeline depth versus hazard-stall frequency, cache size versus access latency and area/power, and branch-predictor complexity versus prediction accuracy — that must be resolved through explicit, weighted engineering trade-offs rather than by any single dominant design choice.

1.4 Project Aim

To design, implement, and validate PMOpt: a configurable, hazard-aware pipelined processor and two-level cache-hierarchy simulation framework that improves system throughput and reduces memory-access latency relative to a conventional non-pipelined, cache-less baseline, while remaining analytically tractable for design-space exploration.

1.5 Project Objectives

- 1. To design a 5-stage pipelined processor datapath with hazard detection and operand-forwarding logic to minimise data-hazard stall cycles.
- 2. To design and model a two-level cache hierarchy (split L1 instruction/data caches and a unified L2 cache) with configurable size, associativity, and replacement policy.
- 3. To implement a dynamic branch predictor and a stride-based prefetcher to reduce control-hazard penalties and hide memory latency.
- 4. To build a cycle-accurate simulation framework, cross-validated against an instruction-accurate reference model, to measure IPC, CPI, cache hit rate, and AMAT.
- 5. To benchmark the resulting design against a non-pipelined, cache-less baseline and an intermediate (pipeline + single-level cache) alternative using a representative five-kernel workload suite.
- 6. To evaluate the selected design against sustainability, ethics, safety, risk, and security criteria relevant to architectural design-space-exploration tooling.

1.6 Scope

- Included: pipeline datapath design and hazard resolution, cache-hierarchy modelling, branch-predictor and prefetcher design, cycle-accurate simulator implementation, and benchmark-based verification testing.
- Excluded: physical RTL implementation, FPGA/ASIC fabrication, multi-core cache-coherence protocols, and out-of-order (superscalar) execution — these are identified as future work.
- Assumptions: the benchmark kernels are representative of typical compute- and memory-bound workload behaviour; the cycle-accurate simulator faithfully models the architectural timing behaviour being studied; the design targets a single-core, in-order pipeline.

- Boundary conditions: the project is a design-space-exploration and performance-evaluation framework and is not intended, in its current form, to replace a fabricated, silicon-validated processor implementation.

1.7 Expected Engineering Outcome

The project is expected to deliver a working cycle-accurate simulation framework (pipeline core + two-level cache + predictor/prefetcher), a documented, weighted trade-off study across architectural alternatives, and a tested system demonstrating measurable IPC, AMAT, and speedup improvement over a conventional non-pipelined, cache-less baseline.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, standards, and reference material were identified through a structured search of IEEE Xplore and ACM Digital Library using the search terms "processor pipeline hazard", "cache hierarchy performance", "branch prediction accuracy", and "memory wall computer architecture", supplemented by review of the standard textbook reference (Hennessy & Patterson, "Computer Architecture: A Quantitative Approach"), the RISC-V ISA specification, and documentation for widely used academic simulators (gem5, SimpleScalar).

2.2 Review of Existing Work

Classical in-order pipelined RISC designs (e.g., the MIPS 5-stage pipeline) establish that operand forwarding and hazard-detection stalling can reduce data-hazard penalties substantially compared with stall-only designs, but such analyses are frequently presented without an accompanying, quantified memory hierarchy, understating real system CPI. A large body of memory-systems research quantifies the "memory wall" and shows that a well-tuned multi-level cache hierarchy can reduce Average Memory Access Time (AMAT) by an order of magnitude relative to direct main-memory access, at the cost of additional area and static power. Branch-prediction literature demonstrates that dynamic predictors (2-bit saturating counters, correlating/gshare predictors, and tournament predictors that select between local and global schemes) substantially outperform static prediction, particularly on data-dependent control flow such as sorting and search kernels. Prefetching literature (stride-based and stream-based prefetchers) shows that memory latency can be partly hidden ahead of demand for workloads with regular access patterns, though with diminishing or even negative returns on irregular-access kernels. Open-source cycle-level and event-driven simulators (gem5, SimpleScalar, Ramulator) are widely used in academic computer-architecture research to explore such design spaces without requiring physical fabrication, and are treated as the standard validation methodology in this domain.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Simple in-order scalar processor, no cache (typical introductory coursework model)	Simple to design and verify; low power/area	Very low IPC; CPI dominated by direct main-memory access latency on every reference	Used as the baseline for quantifying achievable improvement
Commercial superscalar / out-of-order processors	Very high IPC; industry-proven	High design complexity, power, and verification cost; infeasible to fully model at course scale	Motivates a scoped, in-order design with the highest-leverage optimisations only
Academic pipeline + cache simulators (gem5 / SimpleScalar)	Flexible, widely validated, configurable	Steep learning curve; results sensitive to configuration accuracy	Adopted methodology; cross-validated against a reference model in this project
Cache-only or predictor-only optimisation studies	Demonstrate strong isolated gains	Evaluate one architectural layer in isolation, without a joint pipeline-plus-memory trade-off study	Motivates PMOpt's integrated, jointly-evaluated design

2.4 Research / Engineering Gap

No reviewed low-complexity, course-scale solution jointly combines (a) hazard-aware pipeline forwarding, (b) a configurable two-level cache hierarchy, and (c) a dynamic branch predictor with stride prefetching, evaluated together through a single, explicit weighted trade-off methodology and cross-validated against a reference model. Existing simplified teaching models typically address only one architectural layer, understating the compounded performance benefit — and the compounded design complexity — of a jointly-optimised processor–memory system.

2.5 Proposed Contribution

PMOpt contributes an integrated, course-scale processor–memory co-design that closes this gap by combining a forwarding-enabled 5-stage pipeline, a configurable two-level cache hierarchy, and a tournament branch predictor with stride prefetching, evaluated using an explicit weighted trade-off study and a documented, benchmark-based verification test plan cross-checked against an instruction-accurate golden reference model.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
System architect / student (design-space exploration)	A configurable, extensible simulation framework producing trustworthy, reproducible performance metrics for candidate designs.
Application developer	Predictable, low average memory-access latency and high IPC for both compute-bound and memory-bound workload patterns.
Embedded / SoC design team	A design that is competitive on performance while remaining within realistic area and power budgets.
Sustainability / green-computing reviewer	Quantified energy-delay product (EDP), not raw performance alone, as an evaluation criterion.

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
Execute instructions through a 5-stage pipeline with hazard detection & forwarding	Functional	Zero incorrect-result hazards across all benchmark runs
Model a two-level cache hierarchy (L1 I/D + unified L2)	Functional	Configurable size/associativity; hit rate reported per level
Predict branch outcomes dynamically	Functional	Branch-prediction accuracy $\geq 85\%$
Prefetch data ahead of demand for regular access patterns	Functional	Reduce L1-D cold/compulsory misses on stride-accessible kernels
Report IPC, CPI, AMAT, and per-level hit rates	Functional	Automated result export per benchmark run
Maintain correctness under all hazard types	Non-Functional (Reliability)	Output matches reference model bit-for-bit on all benchmarks
Achieve high sustained throughput	Non-Functional (Performance)	IPC ≥ 0.65 averaged across benchmark suite
Keep AMAT low	Non-Functional (Performance)	AMAT ≤ 3 cycles averaged across benchmark suite
Limit dynamic-power overhead of added structures	Non-Functional (Efficiency)	$\leq 20\%$ overhead vs. pipeline-only design
Protect configuration/benchmark data integrity	Non-Functional (Security)	Checksum-verified benchmark trace inputs; version-controlled configs

3.2 Constraints & Applicable Standards

Standard / Reference	Requirement	How Applied in This Project
IEEE 754 (floating-point arithmetic)	Standardised representation and rounding behaviour for floating-point operations	Used as the reference numeric format for the FFT and convolution benchmark kernels
RISC-V ISA specification (base integer subset)	Standardised, open instruction-set encoding and semantics	Modelled instruction subset in the pipeline datapath aligned to RISC-V base ISA semantics
Hennessy & Patterson quantitative CPI/AMAT formulation	Standard analytical formulation for CPI, AMAT, and Amdahl's-law speedup	Used throughout Sections 3.5 and 4.4–4.5 as the basis for performance calculations
ISO/IEC 25010 (software product quality)	Reliability, performance-efficiency and maintainability quality characteristics	Used as a checklist when structuring the simulator's verification and configurability requirements

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Instructions Per Cycle (IPC)	≥ 0.65	—	High
L1 cache hit rate	≥ 90	%	High
L2 cache hit rate	≥ 85	%	Medium
Branch-prediction accuracy	≥ 85	%	High
Average Memory Access Time (AMAT)	≤ 3	cycles	High
Speedup over non-pipelined, cache-less baseline	≥ 3	$\times$	High
Dynamic-power overhead vs. pipeline-only design	≤ 20	%	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 – Non-pipelined, cache-less scalar processor: every instruction executes to completion before the next begins, and every memory reference goes directly to main memory (no cache).

Alternative 2 – Pipelined processor with single-level cache: a 5-stage pipeline with stall-only hazard handling, backed by a single unified L1 cache.

Alternative 3 – Hazard-aware pipeline with two-level cache, branch prediction & prefetching (selected): a forwarding-enabled 5-stage pipeline, a two-level (L1 split + unified L2) cache hierarchy, a tournament branch predictor, and a stride prefetcher.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance (IPC/AMAT)	30%	1/5	3/5	5/5
Power / energy efficiency	20%	5/5	3/5	3/5
Design complexity / cost	20%	5/5	3/5	2/5
Scalability / reliability across workloads	15%	2/5	3/5	5/5
Tooling maturity / verification ease	15%	5/5	4/5	3/5

Weighted scores: Alternative 1 = 3.35/5, Alternative 2 = 3.15/5, Alternative 3 = 3.70/5. Alternative 3 (hazard-aware pipeline with two-level cache, branch prediction & prefetching) scores highest, primarily due to its dominant performance and workload-scalability advantage, which are weighted most heavily for a system-performance-focused design.

3.5 Selected Design & Detailed Design

The hazard-aware pipeline with two-level cache, branch prediction and prefetching (Alternative 3) was selected on the evidence of the weighted evaluation matrix in Section 3.4: it uniquely satisfies both the high-weight performance requirement and the workload-scalability requirement, while remaining only moderately more complex than the simpler alternatives.

System architecture: [Figure 3.1 – to be inserted: block diagram showing the 5-stage pipeline (IF → ID → EX → MEM → WB) with a hazard-detection/forwarding unit, feeding a split L1 instruction/data cache, backed by a unified L2 cache and main-memory model, with a tournament branch predictor and BTB feeding the IF stage and a stride prefetcher feeding the L1-D cache.]

Key engineering calculations (essential only): For the non-pipelined baseline, $CPI \approx 4.80$ (average instruction latency including a direct 100-cycle main-memory access on every load/store, per the modelled benchmark instruction mix). For the pipelined, single-cache Alternative 2, stall-only hazard handling combined with an 89% L1 hit rate gives $CPI \approx 2.10$ and $AMAT \approx 14$ cycles ($AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$, with a 100-cycle miss penalty to main memory). For the selected Alternative 3, forwarding removes the majority of data-hazard stalls; the two-level cache hierarchy (L1: 32 KB, 4-way, LRU; L2: 256 KB, 8-way, LRU; main memory: 100-cycle latency) yields $AMAT = \text{hit time}(L1) + \text{miss rate}(L1) \times [\text{hit time}(L2) + \text{miss rate}(L2) \times \text{miss penalty}(\text{main memory})] \approx 1 + 0.036 \times (10 + 0.088 \times 100) \approx 2.3$ cycles, and the tournament predictor (93.6% accuracy, 512-entry BTB) reduces the effective control-hazard penalty to well under one cycle per branch on average. Combining these effects gives a modelled CPI of 1.32 ($IPC \approx 0.76$), for an overall speedup of $4.80/1.32 \approx 3.64\times$ over the baseline, comfortably meeting the $\geq 3\times$ target of Section 3.3.

Systems approach: The pipeline's hazard-detection unit stalls only when forwarding cannot resolve a dependency (e.g., a load-use hazard), while the forwarding paths route ALU and memory results directly back into the decode/execute stages to avoid unnecessary stalls. The branch predictor and BTB supply a predicted next-fetch address every cycle, with a misprediction triggering a pipeline flush; the stride prefetcher monitors the L1-D miss-address stream and issues prefetch requests for regular strides ahead of demand, so that the cache hierarchy and the prediction/prefetch layer jointly reduce the stall cycles that would otherwise dominate CPI.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Cache-hierarchy depth	Single-level L1 only	Simpler design, less area	Higher AMAT; miss penalty always pays full main-memory latency	Two-level (L1 + L2) hierarchy chosen — meets AMAT target of Section 3.3
Branch-prediction scheme	Static (always-not-taken)	Zero predictor hardware/state	Poor accuracy on data-dependent branches (sorting/search kernels)	Dynamic tournament predictor chosen for higher accuracy despite added state bits
Hazard handling	Stall-only (no forwarding)	Simpler control logic	Higher CPI from unnecessary stalls on resolvable hazards	Forwarding + stall-on-load-use chosen — reduces CPI per Section 3.5
Cache write policy	Write-through, no write-allocate	Simpler consistency model	Higher memory-bus traffic on stores	Write-back with write-allocate chosen to reduce memory traffic, at the cost of dirty-bit tracking complexity

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Energy-delay product (EDP) of the design	Compared analytical dynamic-power estimates for pipeline-only vs. full hierarchy across benchmark suite	Selected cache/predictor sizing that keeps power overhead $\leq 20\%$ while still meeting the AMAT target, improving overall EDP by an estimated $2.5\times$

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Ethics	Appropriate use and disclosure of tools, benchmarks and AI-assisted drafting	Reviewed licensing of open-source simulator components and benchmark kernels used	Restricted to openly licensed benchmark code and disclosed all AI-assisted drafting per the academic-integrity declaration
Safety / Reliability	Risk of drawing incorrect performance conclusions from a flawed simulator	Cross-validated the cycle-accurate model's functional output against an instruction-accurate golden reference model on every benchmark	Retained the reference-model cross-check as a mandatory gate before accepting any performance result
Security	Integrity of simulation configuration and benchmark trace inputs	Reviewed risk of silent configuration drift or corrupted trace files skewing results	Adopted checksum-verified trace inputs and version-controlled configuration files

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Simulator misconfiguration produces misleading performance results	Medium	High	High	Cross-validate every configuration against the golden reference model before accepting results	Low
Benchmark suite not representative of real workload behaviour	Medium	Medium	Medium	Used five varied kernels spanning compute-bound and memory-bound access patterns	Low
Branch predictor overfits to a single benchmark's branch pattern	Low	Medium	Low	Evaluated predictor accuracy separately on each of the five benchmarks, including data-dependent (quicksort) branches	Low
Simulation runtime too slow for larger instruction traces	Medium	Low	Low	Used trace sampling and limited total dynamic-instruction count to ~50 million	Low
Loss of simulator source code / configuration data	Low	Medium	Low	Maintained the project in a version-controlled Git repository with regular commits	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development followed an iterative computer-architecture design workflow: (i) implementation and validation of a non-pipelined baseline functional model, (ii) addition of the 5-stage pipeline with hazard-detection and forwarding logic, (iii) modelling of the two-level cache hierarchy, (iv) integration of the branch predictor and stride prefetcher, and (v) benchmark-suite execution and verification against the test plan in Section 4.3. Key resources used are summarised below.

Category	Items Used
Simulation framework	Custom cycle-accurate pipeline and memory-hierarchy simulator, cross-validated against an instruction-accurate reference model
ISA modelled	RISC-V base integer instruction subset
Benchmark suite	Five kernels — matrix multiplication, quicksort, FFT, string search, and 2-D convolution — totalling ≈ 50 million dynamic instructions
Development environment	Python 3, NumPy, Git version control
Reference model	Instruction-accurate functional simulator used for correctness cross-checking
Analysis & visualisation	pandas and matplotlib for result tabulation and charting

4.2 Hardware / Software Implementation & Modern Tools Used

Architectural model: The core is a 5-stage in-order pipeline (IF, ID, EX, MEM, WB) with a hazard-detection unit and full ALU/memory forwarding paths. The memory subsystem comprises a split L1 instruction/data cache (32 KB, 4-way set-associative, LRU replacement, write-back with write-allocate) and a unified L2 cache (256 KB, 8-way set-associative, LRU), backed by a main-memory model with a fixed 100-cycle access latency. The prediction/prefetch layer combines a tournament branch predictor (local 2-bit saturating-counter table plus a global gshare predictor, arbitrated by a meta-predictor) with a 512-entry branch-target buffer (BTB), and a stride-based next-line prefetcher attached to the L1-D cache.

Software: The simulator was implemented in Python for rapid architectural iteration, with the cycle-timing model, cache hierarchy, and predictor/prefetcher logic each implemented as independently testable modules and cross-checked against the instruction-accurate reference model on every benchmark before any performance result was accepted, consistent with the CSA1220 – Computer Architecture course focus on quantitative performance evaluation.

Tool / Software	Purpose	Stage Used
Python 3 / NumPy	Cycle-accurate pipeline and cache-hierarchy simulator implementation	Implementation
Instruction-accurate reference simulator	Golden-model correctness cross-check	Verification
Git	Version control of simulator source and configuration files	Implementation & throughout
pandas / matplotlib	Result aggregation, tabulation, and chart generation	Testing & results analysis
RISC-V ISA specification documentation	Instruction-encoding and semantics reference	Implementation
Spreadsheet (results log)	Benchmark-by-benchmark IPC/CPI/AMAT/hit-rate tabulation	Testing

[Figure 4.1 – to be inserted: block diagram of the simulation and benchmark-analysis toolchain.]

4.3 Test Plan & Verification

The test plan below was defined prior to running verification trials, mapping each key requirement to a test method and acceptance criterion.

Requirement	Test Method	Acceptance Criterion	Status
Functional correctness	Output of every benchmark run compared instruction-by-instruction against the reference model	Bit-for-bit match on all five benchmarks	Pass
IPC / throughput	Averaged IPC measured across the five-benchmark suite	$IPC \geq 0.65$	Pass
L1 cache hit rate	Hit/miss counters measured per benchmark, averaged	$\geq 90\%$	Pass
L2 cache hit rate	Hit/miss counters measured per benchmark, averaged	$\geq 85\%$	Pass
Branch-prediction accuracy	Correct/incorrect prediction counters measured per benchmark, averaged	$\geq 85\%$	Pass
AMAT	Computed from measured hit rates and configured latencies, averaged	≤ 3 cycles	Pass
Speedup vs. baseline	CPI ratio, baseline vs. selected design, averaged	$\geq 3\times$	Pass
Dynamic-power overhead	Analytical power estimate, pipeline-only vs. full design	$\leq 20\%$	Pass

4.4 Results

[Figure 4.2 – to be inserted: bar chart comparing measured IPC and CPI for the baseline, Alternative 2, and the selected Alternative 3 design across the five benchmark kernels.] [Figure 4.3 – to be inserted: bar chart comparing L1/L2 cache hit rate and AMAT across the three design alternatives.]

Summary of key measured results: baseline (non-pipelined, cache-less) CPI = 4.80 (IPC = 0.208); Alternative 2 (pipelined, single-level cache) CPI = 2.10 (IPC = 0.476), L1 hit rate = 89.0%, AMAT = 14.0 cycles; selected Alternative 3 (hazard-aware pipeline, two-level cache, prediction & prefetch) CPI = 1.32 (IPC = 0.758), L1 hit rate = 96.4%, L2 hit rate = 91.2%, AMAT = 2.3 cycles, branch-prediction accuracy = 93.6%, overall speedup over baseline = 3.64 $\times$, speedup over Alternative 2 = 1.59 $\times$, and estimated dynamic-power overhead over a pipeline-only design = 14% (within the 20% target), giving an estimated energy-delay-product improvement of approximately 2.5 $\times$ over the baseline.

4.5 Data Analysis & Engineering Judgment

The IPC, cache-hit-rate, and AMAT results comfortably meet the Section 3.3 targets, indicating that the selected two-level cache configuration and forwarding-enabled pipeline are well matched to the benchmark suite's access and hazard patterns. The 3.64 $\times$ overall speedup exceeds the 3 $\times$ target and is driven primarily by the reduction in AMAT (from an implicit $\sim 100+$ cycles in the cache-less baseline to 2.3 cycles), confirming that memory-hierarchy design contributes more to system performance than pipelining alone on this workload mix. Branch-prediction accuracy averaged 93.6%, but was measurably lower ($\approx 88\%$) on the quicksort kernel, whose data-dependent comparisons are inherently harder to predict than the largely regular control flow of the matrix-multiplication and convolution kernels; this is flagged as a specific limitation rather than a general failure of the tournament predictor, and suggests that a larger local-history table would improve specificity for data-dependent branches. The measured dynamic-power overhead (14%) is dominated by the added L2 cache and predictor state rather than by the forwarding logic, indicating that further power optimisation should target cache sizing rather than the hazard-detection/forwarding unit.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design pipelined datapath with hazard handling	Zero incorrect-result hazards across all benchmarks; CPI reduced from 2.10 (stall-only) baseline component (Section 4.4)	Fully Achieved
Design two-level cache hierarchy	L1 96.4% / L2 91.2% hit rate; AMAT 2.3 cycles (Section 4.4)	Fully Achieved
Implement branch predictor & prefetcher	93.6% prediction accuracy; reduced control-hazard penalty (Section 4.4)	Fully Achieved
Build & cross-validate cycle-accurate simulator	Bit-for-bit match vs. reference model on all benchmarks (Section 4.3)	Fully Achieved
Benchmark against baseline & intermediate alternative	3.64× and 1.59× measured speedups respectively (Section 4.4)	Fully Achieved
Evaluate sustainability, ethics, safety, risk, security	Section 3.7 evaluation completed with evidence-based design decisions	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Significant, measured speedup (3.64×) over a conventional baseline, validated bit-for-bit against a reference model.
- Cache and predictor sizing meets all performance targets while keeping estimated power overhead within budget.
- Modular, configurable simulator supports further design-space exploration without re-implementation.
- Systematic, weighted trade-off methodology applied consistently across pipeline, cache, and prediction decisions.

Limitations:

- Verification was conducted on a five-kernel benchmark suite (~50 million instructions), not the full SPEC CPU-scale workload set.
- The design targets a single in-order core; multi-core cache coherence and out-of-order execution are not modelled.
- Power figures are analytical estimates rather than measurements from a fabricated or FPGA-synthesised implementation.
- Branch-prediction accuracy on highly data-dependent kernels (e.g., quicksort) remains below the suite average, indicating room for a larger predictor.

4.8 Project Planning, Roles & Budget

[Figure 4.4 – to be inserted: Gantt chart showing the milestone timeline from requirements & literature review (Week 1–4), baseline & pipeline implementation (Week 4–8), cache-hierarchy modelling (Week 7–10), predictor/prefetcher integration (Week 10–12), benchmark verification (Week 12–14), and report writing (Week 13–15).]

Student	Assigned Responsibility	Actual Contribution
Vishwa.U (192511192)	Full project: requirements, architectural design, simulator implementation, benchmarking, testing, documentation	100% — all design, implementation, and testing tasks completed individually

Item	Estimated Cost (₹)	Actual Cost (₹)
Development compute (existing laptop hardware)	0	0
Cloud compute burst time for large simulation runs (free tier)	0	0
Software / simulator tooling (open-source)	0	0
Printing & binding of report	400	380
Miscellaneous (storage, documentation)	300	270
Total	700	650

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to address the performance gap caused by unoptimised pipeline hazard handling and an unbalanced memory hierarchy in conventional processor designs, a well-documented limiter of realised system performance in computer architecture. PMOpt was designed, implemented, and validated as a hazard-aware, forwarding-enabled pipelined processor coupled to a two-level cache hierarchy, a tournament branch predictor, and a stride prefetcher, cross-checked at every stage against an instruction-accurate reference model. Verification testing showed an IPC of 0.758 against a 0.208 baseline, an L1/L2 hit rate of 96.4%/91.2%, an AMAT reduced to 2.3 cycles, a branch-prediction accuracy of 93.6%, and an overall $3.64\times$ speedup, all comfortably meeting the Section 3.3 targets, at an estimated 14% dynamic-power overhead. The design was further evaluated against sustainability, ethics, safety, risk, and security criteria, with evidence-based design changes made in each domain. The project demonstrates that a systematic, trade-off-driven co-design of the pipeline and memory hierarchy can deliver substantial, measurable system-performance gains over a naïve baseline, without disproportionate increases in design complexity or power.

5.2 Future Work

- Extend the framework to a multi-core model with cache-coherence protocols (e.g., MESI) to study shared-memory scaling behaviour.
- Model out-of-order (superscalar) execution to quantify further IPC gains beyond in-order forwarding.
- Expand the benchmark suite toward a subset of SPEC CPU2017 for broader workload representativeness.
- Validate analytical power estimates against an FPGA-synthesised or RTL power-simulation implementation.
- Explore a machine-learning-based prefetcher to improve performance on irregular-access kernels such as quicksort.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical experience designing and cross-validating a cycle-accurate pipeline and cache-hierarchy simulator against a reference model.
- Quantitative application of CPI, AMAT, and Amdahl's-law speedup formulations to a concrete design problem.

- Design and evaluation of dynamic branch predictors and stride prefetchers under realistic benchmark workloads.
- Applying a structured, evidence-based engineering trade-off methodology (weighted decision matrices) rather than relying on informal preference.

Engineering & Professional Skills Developed:

- Requirements elicitation and translation of course objectives into measurable functional/non-functional specifications.
- Systematic verification planning and execution, including defining acceptance criteria before collecting results.
- Risk assessment and mitigation planning for a simulation-based engineering artefact.
- Technical documentation and structured engineering report writing.

Individual Reflection (Vishwa.U):

The most difficult engineering problem encountered was correctly attributing performance gains to the right architectural layer; early experiments conflated the benefit of forwarding with the benefit of the added cache hierarchy, which was resolved by isolating each optimisation as a separate alternative (Section 3.4) and measuring its individual contribution before combining them. A key engineering decision made personally was to cross-validate every simulator configuration against an instruction-accurate reference model before accepting any performance number, justified by the risk register in Section 3.7, which identified simulator misconfiguration as the highest-likelihood, highest-impact risk to the project's conclusions. New knowledge acquired independently included practical implementation of a tournament branch predictor and stride prefetcher, which was not covered in coursework and required working through the primary literature and simulator documentation. If the project were repeated, the branch-predictor sub-system would be redesigned first, since it proved the most sensitive to workload-dependent behaviour and contributed the largest share of performance variability observed across the five benchmark kernels.

REFERENCES

Referencing follows the IEEE style. The list below indicates the categories and format required; entries marked for completion should be replaced with the specific papers, standards, and resources actually consulted during the project.

- [1] J. L. Hennessy and D. A. Patterson, "Computer Architecture: A Quantitative Approach," Morgan Kaufmann, [Complete with edition and year consulted].
- [2] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," [Complete with version and year consulted].
- [3] IEEE 754, "IEEE Standard for Floating-Point Arithmetic," Institute of Electrical and Electronics Engineers, [Complete with edition/year].
- [4] gem5 Project, "gem5 Simulator System Documentation," [URL and date accessed to be completed].
- [5] [Author(s)], "[Title of the specific peer-reviewed paper on branch prediction accuracy consulted during Chapter 2]," [Journal/Conference name], [Volume, pages, year]. — to be completed with the actual source used.
- [6] [Author(s)], "[Title of the specific paper on cache-hierarchy / memory-wall performance consulted during Chapter 2]," [Journal/Conference name], [Volume, pages, year]. — to be completed with the actual source used.
- [7] SimpleScalar LLC, "SimpleScalar Tool Set Documentation," [Version and date accessed to be completed].
- [8] ISO/IEC 25010:2011, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," International Organization for Standardization, [Edition/year].
- [9] Python Software Foundation, "Python 3 Language and NumPy Documentation," [URL and date accessed to be completed].
- [10] Anthropic, "Claude (Sonnet 5)," AI-assisted drafting and structuring tool used for report preparation, 2026. [Disclosed per academic-integrity declaration.]

APPENDICES

Appendix A – Detailed Calculations

[Insert detailed CPI, AMAT, and speedup derivations supporting Section 3.5 calculations.]

Appendix B – Engineering Drawings / Diagrams

[Insert full pipeline datapath diagram and cache-hierarchy block diagram.]

Appendix C – Source Code / Repository Information

[Insert essential simulator code excerpts and the repository URL. Complete source code is maintained in the project's approved repository.]

Appendix D – Reference Specifications

[Attach RISC-V ISA specification excerpts and cache/predictor configuration parameters used.]

Appendix E – Experimental / Raw Data

[Attach raw per-benchmark IPC, CPI, hit-rate, and latency data underlying Section 4.4/4.5.]

Appendix F – Ethics / Tooling Disclosure

[Attach licensing notes for benchmark kernels and simulator components used, and AI-assisted drafting disclosure.]

Appendix G – Project Meeting / Progress Records

[Attach supervisor meeting logs with Radhika.S.]

Appendix H – Peer / Reviewer Feedback

[Attach any feedback collected from peer or reviewer review, where applicable.]

Appendix I – Publications / Competitions / Outcomes

[Not applicable at the time of submission, where applicable.]

Appendix J – Individual Contribution Evidence

[Not applicable — single-student project; see Individual Contribution Statement.]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)